

lecture 5

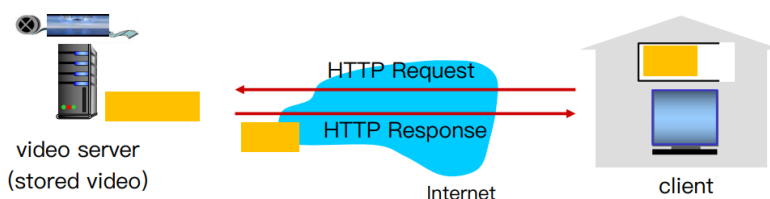
• Video streaming and content distribution networks

- video streaming

- 视频流和CDN:

- 视频流量是互联网带宽的主要消耗者。
 - Netflix和YouTube占据了37%、16%的家庭ISP下行流量。
 - YouTube用户约10亿，Netflix用户约7500万。
 - 挑战1：规模——如何覆盖10亿用户？
 - 单一的超大视频服务器无法满足（为什么？因为瓶颈和延迟问题）。
 - 挑战2：异构性——不同用户设备能力差异（如有线和移动、带宽高和低）。
 - 解决方案：分布式、应用层基础设施。
 - 视频是以固定速率显示的一系列图像（如每秒24帧）。
 - 不同帧率的视频画面（30 FPS、60 FPS、120 FPS），帧数越高，画面越流畅，说明视频的本质是图像连续播放。
 - 编码（压缩）：利用图像内部和图像之间的冗余来减少编码所需的比特数。
 - 空间冗余（图像内部）——空间编码举例：同一种颜色重复时，只需记录颜色和重复次数，而不是每个像素值。
 - 时间冗余（图像之间）——时间编码举例：只发送两帧之间的差异，而不是完整帧。
 - 视频编码码率与分辨率、帧率相关。
 - CBR（固定码率）：视频编码速率固定。
 - VBR（可变码率）：编码速率随着空间和时间编码变化而变化。
 - 数字图像由像素阵列组成，每个像素用bit表示。

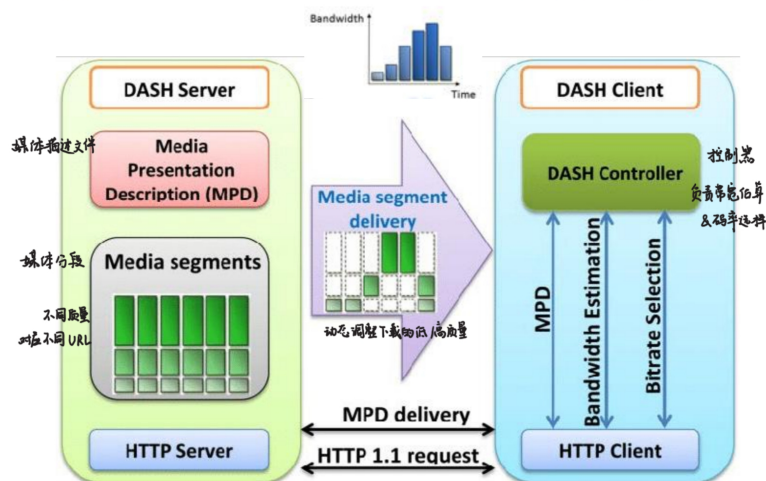
- HTTP Streaming（HTTP流媒体）



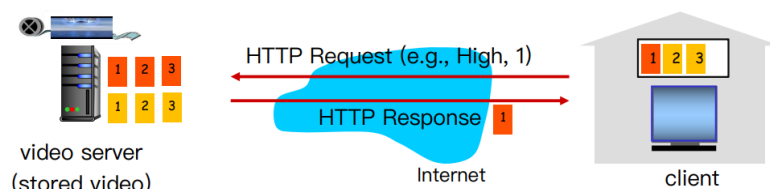
- 视频服务器保存视频，用户（客户端）通过互联网发送HTTP请求，服务器用HTTP响应发送视频数据。
 - 所有客户端收到的视频编码格式都是一样的。
 - 人类用户需求可能不同。
 - 客户端可用带宽可能不同，且带宽可能随时变化。

- 流媒体：DASH (Dynamic Adaptive Streaming over HTTP, 动态自适应流媒体)

Streaming multimedia: DASH

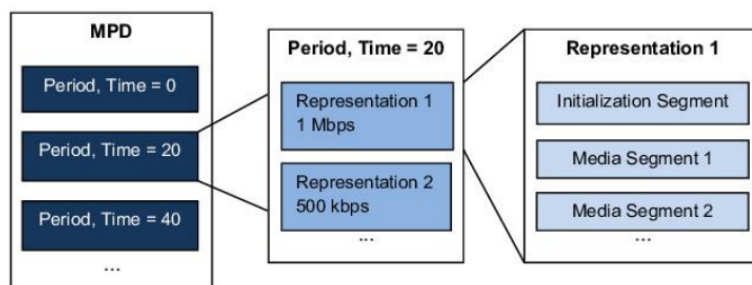


- 服务器端：
 - 将视频文件分成多个块 (chunks)，每个块可以用不同码率编码。
 - 每个块单独存储，编码速率不同。
 - 提供manifest文件，里面有每个块不同码率的URL信息。
- 客户端：
 - 定期测量服务器到客户端的带宽。
 - 根据manifest文件，每次只请求一个块。
 - 根据当前带宽选取最大可持续的编码速率。
 - 可在不同时间点选择不同编码速率（根据带宽变化动态调整）。
 - “智能”在客户端：由客户端决定
 - 何时请求chunk（避免缓冲区枯竭或溢出）
 - 什么编码速率请求（带宽充足时请求高质量）
 - 从哪里请求chunk（可以选择离自己近或带宽高的服务器URL）
- 客户端发起HTTP请求，获得MPD，再根据带宽估算选择合适码率分段请求。
- MPD交付后，客户端不断请求媒体分段，依据带宽变化调整请求码率。



- 具体DASH请求过程示例：
 - 视频服务器存储不同码率的分段。

- 客户端通过HTTP请求指定分段和码率（如请求高质量1号块）。
- 服务器返回对应分段，客户端逐块拼接成完整视频。



Media presentation description (MPD), also known as manifest

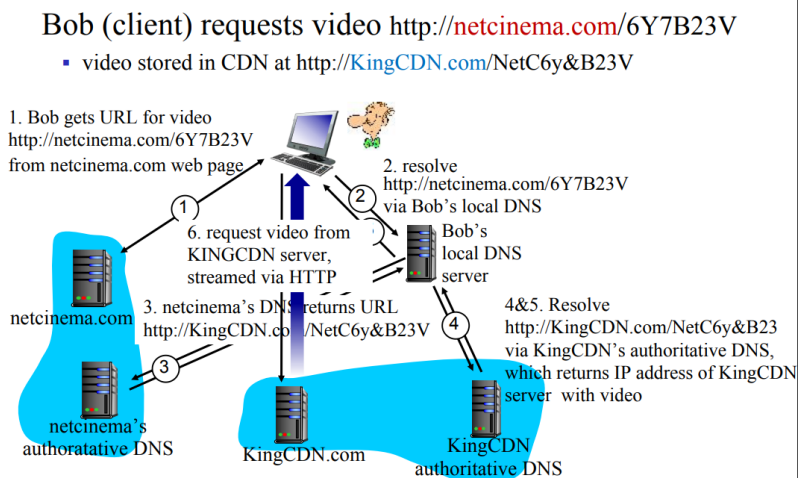
- MPD（媒体描述文件）结构：
 - 不同时间段（Period）有不同表示（Representation），每种表示对应不同码率（如1Mbps、500kbps）。
 - 每种表示有初始化段（Initialization Segment）和具体媒体分段（Media Segment 1、2等）。

• content distribution network **CDN**

- 挑战：如何将内容（从数百万视频中选出）同时流式传输给成千上万用户？
 - 方案1：单一大型“超级服务器” →→→ 总结：这种方案**无法扩展**（doesn't scale）
 - 单点故障风险
 - 流量巨大
 - 路径到远端用户太长
 - 出口链路上发送多份视频内容
 - 方案2：内容分发网络（CDN）
 - 在多个地理分布站点存储/服务多个视频副本
 - **Enter deep**：CDN服务器深入接入网络，部署在ISP内，接近用户（如Akamai，全球1700个点）
 - **Bring home**：在互联网交换点（IXP）部署少量大型集群，位于ISP外部（如Limelight）
- CDN在各节点存储内容副本（如Netflix在CDN节点存MadMen剧集）
- 用户请求内容时：
 - 被导向最近的副本，获取内容
 - 如果网络路径拥堵，可选择其他副本
- 重点是如何应对互联网拥堵，CDN面临的挑战包括：
 - **选择在CDN节点存储哪些内容？**
 - 简单拉取策略：用户请求后才存储内容
 - **从哪个CDN节点获取内容？**

- 集群选择策略 cluster selection strategy
 - 一种常见策略是将客户端分配到“地理上最近”的集群：
 - DNS请求到达时，CDN选择地理上最近的集群
 - 但地理最近不一定网络跳数最少
 - 忽略了延迟和可用带宽随时间变化的影响
 - 另外，CDN还会周期性进行**实时测量**（实时探测）：
 - 每个集群定期向全球的本地DNS发送探测包，测量延迟和丢包表现
 - 但很多本地DNS不响应探测包

• 检索内容的操作流程？



- 展示了用户从一个网站点播视频时，CDN如何通过“DNS引导（重定向）+就近分发”把用户流量导向合适的CDN边缘节点，从而高效地把视频流回给用户。
- 关键思路：网页给的是内容站点URL，但视频实际存储在CDN域名下。通过两轮DNS解析，把用户引导到“合适的CDN服务器IP”，然后用HTTP把视频分段/字节流回来。
- 图中参与角色（图例对照）
 - Bob (client)：发起请求的客户端。
 - netcinema.com：内容网站（“源站”/Origin）及其权威DNS（authoritative DNS）。
 - Bob's local DNS（LDNS）：用户网络中的本地递归DNS。
 - KingCDN.com：CDN提供商的域名与其权威DNS、以及具体承载视频的CDN边缘服务器。
- 图中文字与流程的对应步骤
 - 页面上方文字
 - “Bob (client) requests video <http://netcinema.com/6Y7B23V>”：用户在netcinema的网页上点开视频，初始URL属于内容站点域名。
 - “video stored in CDN at <http://KingCDN.com/NetC6y&B23V>”：实际视频文件放在CDN域名下（KingCDN.com），也就是内容托管在CDN。

- 编号步骤
 - Bob从netcinema网页上获得视频URL (<http://netcinema.com/6Y7B23V>)。
 - Bob把netcinema.com/6Y7B23V交给本地DNS (LDNS) 去解析 (正常的DNS解析流程)。
 - netcinema的权威DNS返回一个“指向CDN的URL” (常见实现是返回CNAME或在网页中直接给出CDN地址)，例如 <http://KingCDN.com/NetC6y&B23V>。
 - 含义：把内容交由CDN分发，下一步要解析CDN域名。4 & 5. 解析 KingCDN.com/NetC6y&B23V：LDNS去询问KingCDN的权威DNS。CDN权威DNS根据策略 (如地理位置、测得的RTT/丢包、服务器负载等) 选择一个“最合适的CDN边缘服务器”，并返回其IP地址。
 - Bob用获得的CDN边缘服务器IP，发起HTTP请求 (GET)，视频以HTTP方式流回客户端 (可能是分段/分片或按字节范围返回)。
- 为什么要这么做 (隐含逻辑)
 - 通过DNS“导流”到CDN边缘：
 - 就近访问：降低时延、提升带宽利用率与质量稳定性。
 - 分担源站压力：大量并发被CDN节点吸收，源站只需回源少量请求 (缓存未命中时)。
 - 动态选择：CDN权威DNS可依据实时测量与策略把不同地区/不同时间的用户分配到不同节点，提升整体服务质量。
- 图中细节/术语补充 (考试常问)
 - LDNS (Local DNS)：用户侧递归DNS，替用户完成解析并缓存结果 (受TTL影响)。
 - Authoritative DNS (权威DNS)：负责最终回答其管理域名记录的DNS服务器。
 - CNAME 方式引流：常见做法是内容域名解析到CDN域名 (别名记录)，再解析CDN域名到实际边缘节点IP。
 - 集群选择策略：地理最近并不等于网络路径最优，因此CDN会结合实时测量 (RTT、丢包、带宽可用性) 与负载做决策。
 - 回源与缓存：
 - 若边缘节点缓存命中：直接回内容给Bob。
 - 若未命中：边缘节点向上级或源站拉取，再缓存并回给Bob (后续用户可命中)。
 - 与DASH/自适应的关系：
 - 第6步“HTTP流回”在现代平台往往对应“按分段 (segment) 逐个拉取”，客户端依据带宽估计选择码率 (ABR)。CDN节点只需按URL把相应分段返回即可。
- 可能的变体/实现方式
 - 有的网站在第1步就让网页里的播放URL直接是CDN域名 (省去第2-3的“再指向”过程)。

- 也可用HTTP 302重定向把用户从源站URL跳到CDN URL（和CNAME引流思路类似）。
- 有些CDN采用Anycast或全局负载均衡器，配合DNS策略进一步优化“就近”。
- 考点速记
 - 两类DNS：本地递归DNS（LDNS）与权威DNS（Authoritative DNS）的职责区分。
 - CDN引流关键：用DNS（常见是CNAME）把用户导向“合适的边缘IP”。
 - 选择“合适”的依据：地理位置只是其一，实际更看RTT/丢包/带宽与节点负载的实时测量。
 - 优势：降低时延、提高可用性与扩展性、减轻源站压力、支持自适应码率分段传输。
- 总结
 - 这页图把“从用户点开内容，到被DNS引导到CDN边缘节点，再通过HTTP把视频流回来”的全过程用6个步骤串起来了。核心是“DNS驱动的就近内容分发”，为大规模流媒体提供低时延、高吞吐、可扩展的传输路径。

• Socket programming with UDP and TCP

- 详见obsidian笔记

• Chapter 2 Summary

- 应用架构
 - 客户端-服务器（client-server）：服务器提供服务，客户端发起请求。如HTTP网页、邮件服务等。
 - 点对点（P2P）：每个节点既是客户端也是服务器。例如BT下载、Skype等。
- 应用服务需求
 - 可靠性：数据是否能完整、准确送达。
 - 带宽：每秒可传输的数据量。
 - 时延（delay）：数据从发送到接收的总耗时。
- 互联网传输服务模型
 - TCP：面向连接、可靠（connection-oriented, reliable）。
 - UDP：不可靠、数据报（unreliable, datagrams）。
- 具体协议
 - HTTP（网页）、SMTP/POP/IMAP（邮件）、DNS（域名解析）、BitTorrent（P2P文件分享）。
- 视频流与CDN
 - 流媒体和内容分发网络，提升访问速度与体验。
- Socket编程
 - TCP、UDP socket接口的使用。

- 典型请求-应答消息交换流程
 - 客户端发起请求，要求信息或服务。
 - 服务器响应，返回数据和状态码。
 - 常见于HTTP、DNS、邮件等协议。
- 消息格式
 - headers（首部）：描述数据的附加信息，比如类型、长度、编码方式。
 - data（数据）：实际要传递的信息内容。
- 重要主题（protocol设计的三组对比）
 - 中心化 vs 分布式：如传统网页服务 vs P2P下载。
 - 无状态 vs 有状态：无状态如HTTP（每次请求独立），有状态如FTP、TCP连接（需维护会话）。
 - 可靠 vs 不可靠消息传输：如TCP与UDP的本质区别。